Update on form inference model

Contents

Update on form inference model 1
The context of the project 1
The problem of the form structure inference 1
The strategy 2
Current prototype 3
Represent analysis as a term 3
Example of Rewriting rules 3
model output 4
Identifying presentation 4
ex 2.11 4
ex 2.12 5
Identifying compound themes 6
ex 6.1 6
ex 6.2 7
ex 6.5 9

The context of the project

We break the overarching form inference (model : piece at the note level → FormTree) into 3 parts

- harmonic reduction model : piece at the note level → harmony and cadence label
  - example output: `[I, V(HC), IV, V, I(PAC)]`
- motivic repetition model : piece at the note level → material markers
  - example output: `["a","b","c","a","b","c","d","e","a","b","c"]`
- **form structure model : (material marker, harmony, cadence) → FormTree**
  - example input:

    ```
    {a ⟦I-V None⟧} + {b ⟦I-III None⟧} + {a ⟦VI-II None⟧} + {b ⟦VI-III None⟧}
    ```

    we want to say that this is the same idea expressed in different harmonic context
  - example output:

    ```
    1 Presentation <b.i. := "a" + "b" >
    2       (idea usedOn ⟦I-V None⟧+⟦I-III None⟧)
    3       (idea usedOn ⟦VI-II None⟧+⟦VI-III None⟧)
    ```

The problem of the form structure inference

We want to describe the input in terms of formal units and variable binding in a compact way

- Formal function imposes constraints on what kind of analysis is possible
  - The term `(Presentation ideaBinding unit1 unit2)` entails that
    - unit1 begins on I and
    - the idea of unit1 and unit2 are the same
- Description length impose a ranking on alternative analysis (so the model should prefer the third instead of the first).
  - in general we can costs on the "operations" e.g. make "catch-all" merge rule such as `Concatenate` costly so the model will prefer to minimize their usage

- the trivial description

```
1     = {a ⟦I-V None⟧} + {b ⟦I-III None⟧} + {a ⟦VI-II None⟧} + {b ⟦VI-III
  None⟧}
```

- a short description

```
1     = (idea usedOn ⟦I-V None⟧+⟦I-III None⟧) + (idea usedOn ⟦VI-II None⟧+⟦VI-
  III None⟧)
2         where idea := "a" + "b"
```

- a short description using formal functions

```
1     = Presentation <b.i. := "a" + "b" >
2         (idea usedOn ⟦I-V None⟧+⟦I-III None⟧)
3         (idea usedOn ⟦VI-II None⟧+⟦VI-III None⟧)
```

The strategy

- we represent form analysis as a term (of a recursive type).
- [mostly music-theoretical] we define term rewriting rules encoding music theoretical knowledge on form.
- we search for the smallest equivalent term for the input (reachable by the collection of rewrite rules). [mostly mathematical & programming]

Current prototype

Represent analysis as a term

Goals:

- we want to be able to recover the original surface (lossless encoding)
- we want to factor out formal repeats (compressible)
- we want the AST to be recursive, sentence and period should take FormalUnit as arguments (recursive)

The main AST

```hs
data FormalUnit
  = Leaf Idea PhraseParam
  | RefFormalUnit String
  | UseIdeaWith Idea PhraseParam
  | Sentence IdeaBinding FormalUnit FormalUnit
  | Period FormalUnitBinding FormalUnit FormalUnit
  | Presentation IdeaBinding FormalUnit FormalUnit
  | Continuation FormalUnit FormalUnit
  | Concatenate FormalUnit FormalUnit
  | ...

```

Auxillary definitions

```
data CadenceStrength = Weak | Strong | None
data Harmony = I | II | III | IV | V | VI | VII
data Idea
  = Label String
  | Concat Idea Idea
  | RefIdea String

data PhraseParam
  = Param [Harmony] CadenceStrength
  | ConcatParam PhraseParam PhraseParam

data IdeaBinding = BindIdea String Idea
data FormalUnitBinding = BindFormalUnit String FormalUnit
```

Note that FormalUnit is now a recursive AST

Example of Rewriting rules

```
premises
      match = Concatenate x y
      IdeaOf x = IdeaOf y
      StartingHarmonyOf x = I

conclusions
      match ~>
          (Presentation
              (BindIdea "b.i." (IdeaOf x))
              (UseIdeaWith (RefIdea "b.i.") (GetParam x))
              (UseIdeaWith (RefIdea "b.i.") (GetParam y))
          )
```

model output

Identifying presentation

ex 2.11



We distinguish between bindings for idea and bindings for formal unit binding. For the presentation, it is the idea that is repeats, while for the period the repeating thing is the formal unit (ABAC)

```
Input surface:
  {a ⟦I None⟧} + {a ⟦IV None⟧}



Output analysis:
  Presentation
    <"b.i." := "a">
    UseIdeaWith   "b.i." ⟦I None⟧
    UseIdeaWith   "b.i." ⟦IV None⟧


  {"a" ⟦I None⟧} + {"a" ⟦IV None⟧}
```

This example illustrates the basics of how variable binding leads to smaller size. By default, we assign a high cost of raw repetition markers from the input (as they are placeholders for the surface motivic ideas). On the other hand, calling a binding variable just cost 1.

ex 2.12



This time we make the input a little bit harder in that the basic idea is not an atomic marker, but the concatenation of two. And the model need to identify the repetition of “a”+“b” as the basic idea, and correctly build up the phrase parameter for the concatenated phrases.

```
 1 Input surface:
 2   {a ⟦I-V None⟧} + {b ⟦I-III None⟧} + {a ⟦VI-II None⟧} + {b ⟦VI-III None⟧}
 3
 4
 5
 6
 7 Output analysis:
 8   Presentation
 9     <"b.i." := "a"+"b">
10     UseIdeaWith   "b.i." ⟦I-V None⟧+⟦I-III None⟧
11     UseIdeaWith   "b.i." ⟦VI-II None⟧+⟦VI-III None⟧
12
13
14
15   {"a" ⟦I-V None⟧}
16   +
17   {"b" ⟦I-III None⟧} + {"a" ⟦VI-II None⟧} + {"b" ⟦VI-III None⟧}
```

Identifying compound themes

ex 6.1




```
Input surface:
  {a ⟦I-V None⟧}
  +
  {b ⟦I-V-I-V Weak⟧}
  +
  {c ⟦IV-V-I None⟧}
  +
  {d ⟦VI-VII-I-V Weak⟧}
  +
  {a ⟦I-V None⟧} + {b ⟦I-V-I-V Weak⟧} + {c ⟦V None⟧} + {e ⟦I Strong⟧}



Output analysis:
  Period
    <<"b.i." := Continuation   {"a" ⟦I-V None⟧} {"b" ⟦I-V-I-V Weak⟧}>>
    "b.i." + Continuation   {"c" ⟦IV-V-I None⟧} {"d" ⟦VI-VII-I-V Weak⟧}
    "b.i." + Continuation   {"c" ⟦V None⟧} {"e" ⟦I Strong⟧}



  Period
    <<"b.i." := {"a" ⟦I-V None⟧}>>
    "b.i."
    +
    {"b" ⟦I-V-I-V Weak⟧}
    +
    Continuation   {"c" ⟦IV-V-I None⟧} {"d" ⟦VI-VII-I-V Weak⟧}
    "b.i."
    +
    {"b" ⟦I-V-I-V Weak⟧} + Continuation   {"c" ⟦V None⟧} {"e" ⟦I Strong⟧}
```

ex 6.2




```
Input surface:
  {a ⟦I None⟧}
  +
  {b ⟦I None⟧}
  +
  {c ⟦I None⟧}
  +
  {a ⟦V None⟧}
  +
  {b ⟦V None⟧}
  +
  {c ⟦V None⟧}
  +
  {b ⟦I-VI None⟧} + {b ⟦IV-II None⟧} + {b ⟦V None⟧} + {d ⟦I-II-V-I Strong⟧}



Output analysis:
  Continuation
    Presentation
      <"b.i." := "a"+"b"+"c">
      UseIdeaWith   "b.i." ⟦I None⟧+⟦I None⟧+⟦I None⟧
      UseIdeaWith   "b.i." ⟦V None⟧+⟦V None⟧+⟦V None⟧
    Continuation
      Presentation
        <"b.i." := "b">
        UseIdeaWith   "b.i." ⟦I-VI None⟧
        UseIdeaWith   "b.i." ⟦IV-II None⟧
      Continuation   {"b" ⟦V None⟧} {"d" ⟦I-II-V-I Strong⟧}



  Sentence
    <"aSent" := "a"+"b"+"c">
    Presentation
```

```
37        <"b.i." := "aSent">
38        UseIdeaWith   "b.i." ⟦I None⟧+⟦I None⟧+⟦I None⟧
39        UseIdeaWith   "b.i." ⟦V None⟧+⟦V None⟧+⟦V None⟧
40      Continuation
41        Presentation
42          <"b.i." := "b">
43          UseIdeaWith   "b.i." ⟦I-VI None⟧
44          UseIdeaWith   "b.i." ⟦IV-II None⟧
45        Continuation   {"b" ⟦V None⟧} {"d" ⟦I-II-V-I Strong⟧}
```

ex 6.5




```
 1 Input surface:
 2   {a ⟦I-V None⟧}
 3   +
 4   {a ⟦V-I None⟧}
 5   +
 6   {b ⟦IV-I None⟧}
 7   +
 8   {b ⟦IV-I None⟧} + {c ⟦II None⟧} + {c ⟦I None⟧} + {d ⟦V Weak⟧}
 9
10
11
12
13 Output analysis:
14   Continuation
15     Presentation
16       <"b.i." := "a">
17       UseIdeaWith   "b.i." ⟦I-V None⟧
18       UseIdeaWith   "b.i." ⟦V-I None⟧
19     Continuation
20       {"b" ⟦IV-I None⟧}
21       Continuation
22         {"b" ⟦IV-I None⟧}
23         Continuation
24           {"c" ⟦II None⟧} Continuation   {"c" ⟦I None⟧} {"d" ⟦V Weak⟧}
25
26
27
28   Sentence
29     <"aSent" := "a">
30     Presentation
31       <"b.i." := "aSent">
32       UseIdeaWith   "b.i." ⟦I-V None⟧
33       UseIdeaWith   "b.i." ⟦V-I None⟧
```

```
    Continuation
      {"b" ⟦IV-I None⟧}
      Continuation
        {"b" ⟦IV-I None⟧}
        Continuation
          {"c" ⟦II None⟧} Continuation   {"c" ⟦I None⟧} {"d" ⟦V Weak⟧}
```